Simulating double-precision GEMM with Ozaki-style multi-word arithmetic at Wafer Scale

zlan

September 2026

Key Idea The Ozaki scheme allows higher-precision GEMM to be approximated using lower-precision operations. The key intuition is that

$$A \in \mathbb{R}^{n\times k}; B \in \mathbb{R}^{k\times m}; C \in \mathbb{R}^{n\times m}$$

$$AB = C$$

can be obtained as

$$A = \sum_i A_i; \quad B = \sum_i B_i$$

$$C = \sum_i \sum_j A_i B_j$$

where A_i and B_i are stored in reduced precision, while A, B, and the reconstructed C are stored in higher precision.

A simple two-word split is

$$A, B, C \text{ stored in } \texttt{fp32}$$

$$A_0, A_1, B_0, B_1 \text{ stored in } \texttt{fp16}$$

$$A_0 = \texttt{fp16}(A) \quad A_1 = \texttt{fp16}(A - \texttt{fp32}(A_0))$$

$$B_0 = \texttt{fp16}(B) \quad B_1 = \texttt{fp16}(B - \texttt{fp32}(B_0))$$

$$C = A_0B_0 + A_0B_1 + A_1B_0 + A_1B_1$$

the intermediate products are stored in `fp32` (optionally A_1B_1 can be excluded).

C retains more accuracy than A_0B_0, achieving a quality-up compared to simple low-precision GEMM.

Ozaki converts one `fp32` GEMM into four `fp16` GEMMs and a `fp32` sum reduction.

Performance Considerations On the A100 GPU, `fp32` throughput is 19.5 TFLOPS, while the reduced precision tensor core performance is 624 TFLOPS in `fp16`, which can compensate for the increased number of FLOPs. Moreover, A_1 and B_1 are generally more sparse than A and B, providing benefits for accelerators optimized for sparsity.

-	Nvidia H100 SXM	Cerebras WSE-3
FP16 FLOPS	989 TFLOP/s	∼15.6 PFLOP/s
FP32 FLOPS	67 TFLOP/s	∼7.8 PFLOP/s*
HBM Tput	3.35 TB/s	–
SRAM Tput	–	21 PB/s
MemoryX Tput	–	∼150 GB/s‡

Novel Idea Most dataflow accelerators primarily target `fp16` precision and are typically overlooked for scientific computing because higher-precision arithmetic is limited. A method similar to the Ozaki approach used on Nvidia GPUs could unlock hidden potential on these accelerators. It would be interesting to compare Ozaki on CS-3/CS-4 WSEs with GPU implementations that simulate FP32 and FP64.

An important caveat is that on GPUs `fp64`, `fp32`, and `fp16` are all supported internally, but the use of reduced precision offers a major performance jump compared to single or double precision thanks to the Tensor Cores. This does not hold on Cerebras. First `fp64` is not supported. This causes (1) `fp64` accumulation to have to happen off-chip, and (2) there is no `fp64` baseline to compare with on par of hardware.